

Verifiable Self-Mix

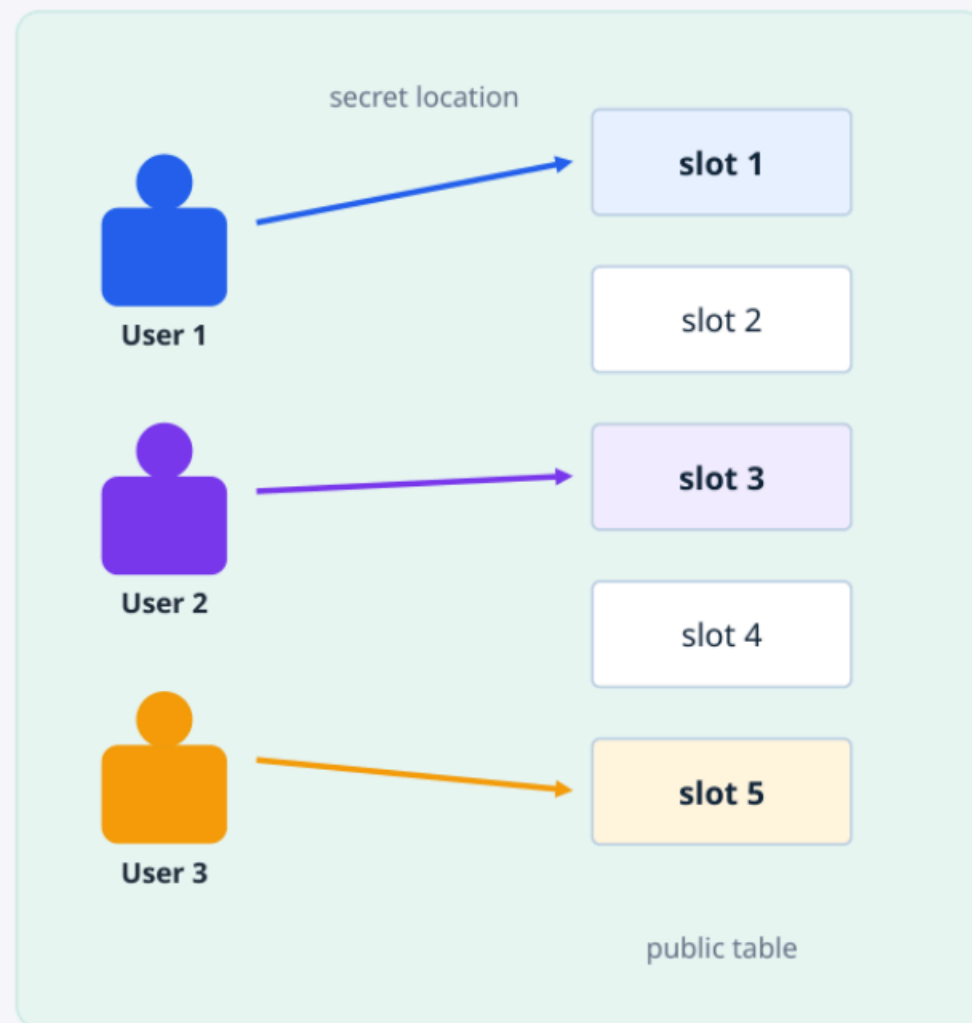
Anonymous communication with
unconditional individual verification

Doron Zarchy

APSIA, SnT, University of Luxembourg

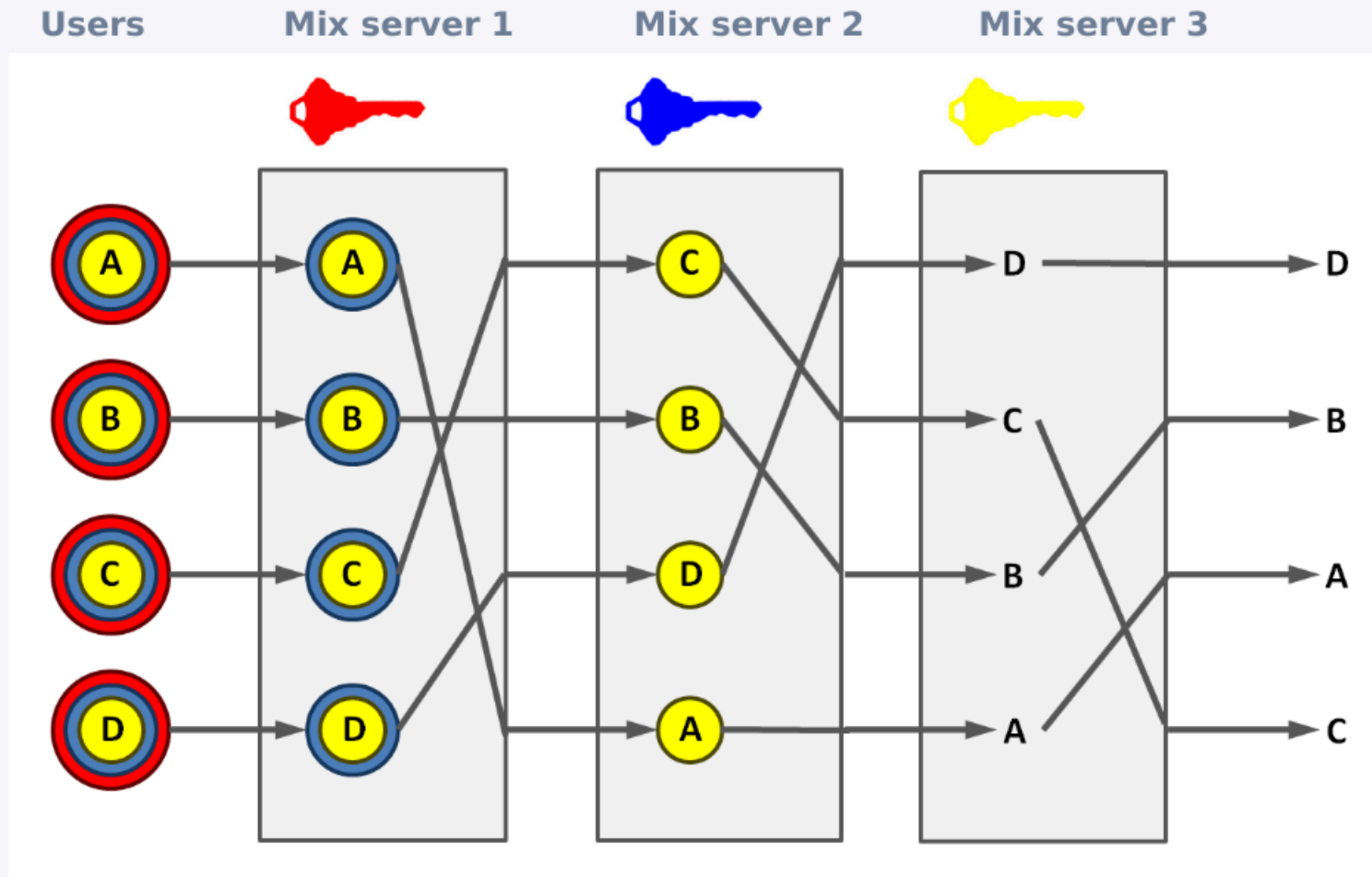
ESORICS 2026 · Rome

Security and Trust Management



Mixnets

Users submit encrypted messages to a sequence of mix servers

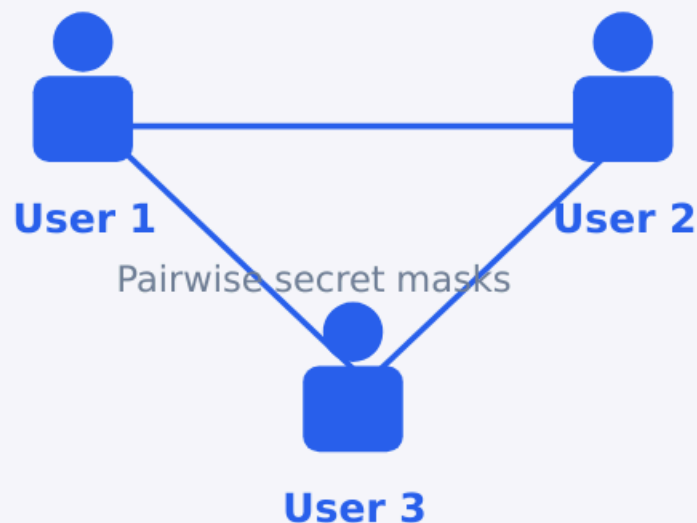


- Each server reshuffles the batch.
- Remove an encryption layer (shown) or re-encrypt.
- Very efficient on the user side.
- Problem: correctness requires trust unless shuffle proofs are verified.

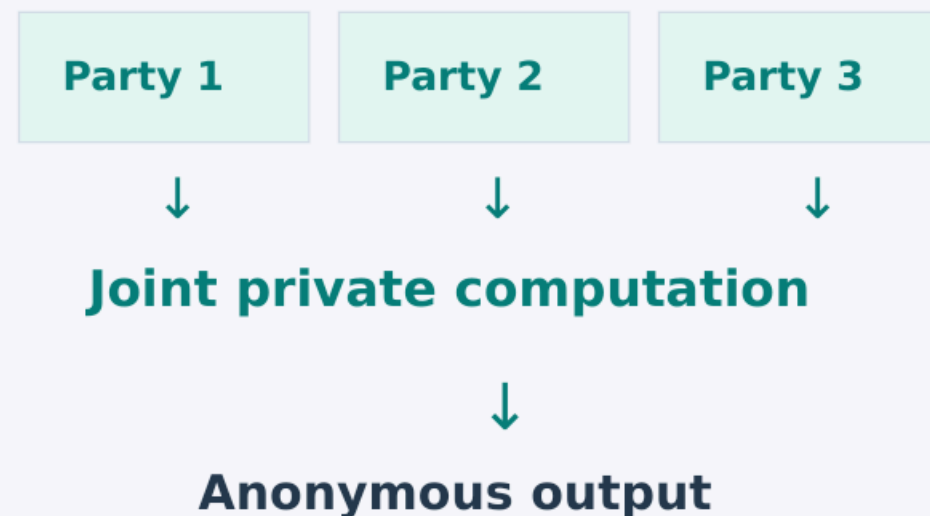
DC-net / MPC style

Participants jointly hide the sender-to-message mapping

DC-net: shared masks cancel



MPC: compute on secret shares



- Distribute trust across participants or computing parties.
- Problem: high communication load.
- Problem: in basic DC-nets, one malicious user can stall execution.

The design tension

Anonymous communication systems optimize different resources

Mixnets

Efficient batch shuffling

- Compact client submissions
- Well-understood cryptographic proofs

Trust: at least one honest mix server

DC-net / MPC style

Less centralized trust

- Can remove a dedicated mixer
- But often more interaction / disruption handling

Cost: communication and coordination

VSM target

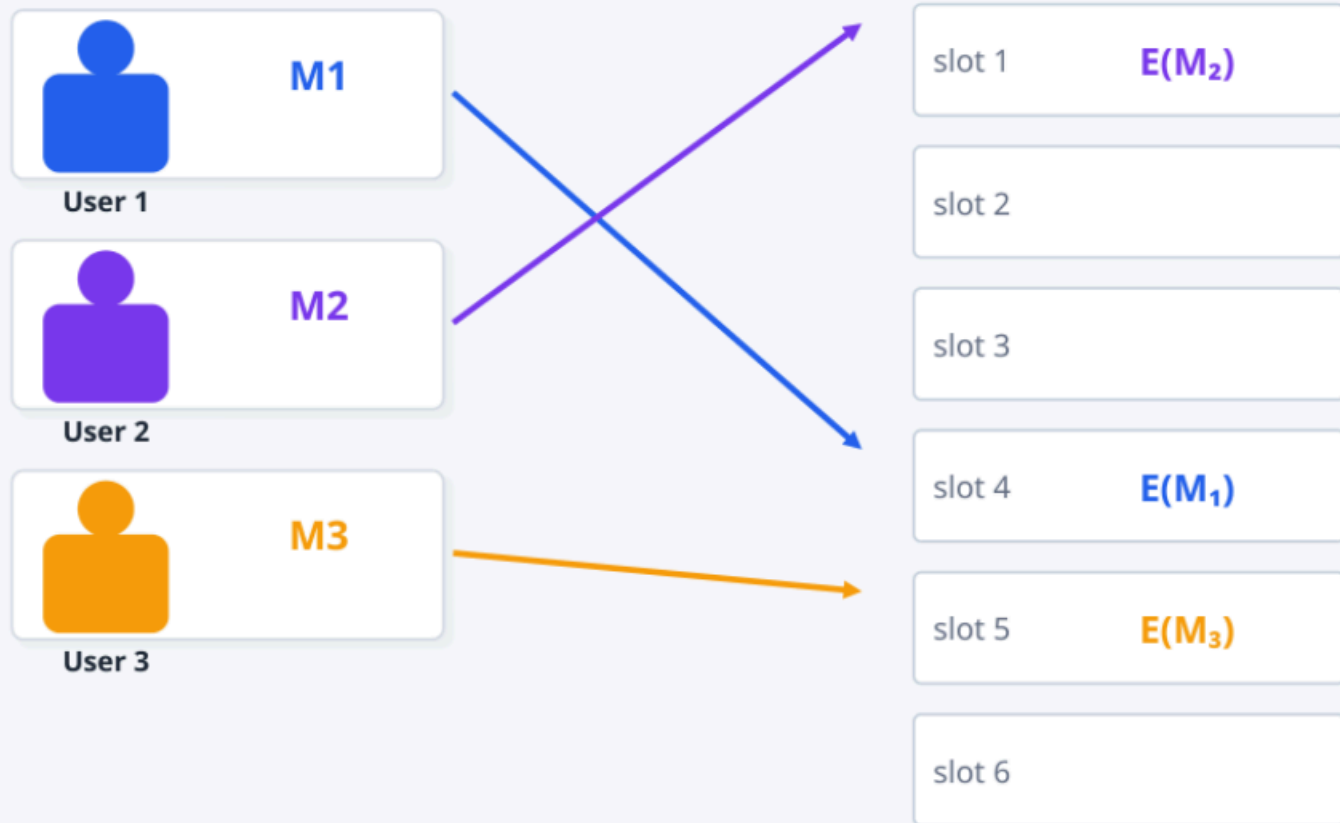
A different point in the space

- Small private trusted state
- Users verify inclusion themselves
- No user-user interaction after preprocessing

Tradeoff, not universal dominance

VSM goal

Give every sender a secret, unique output slot



After optional decryption

User i checks

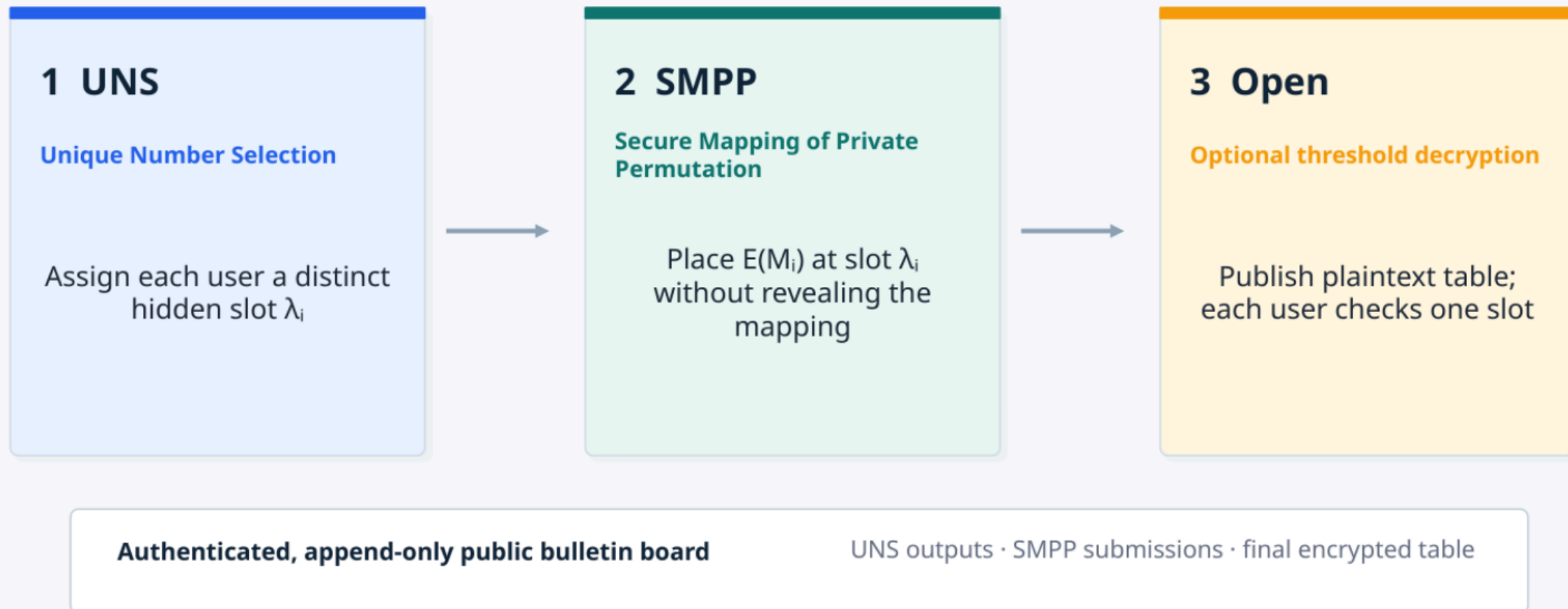
$$T[\lambda_i] = M_i$$

λ_i = user i's private slot
 M_i = user i's message

No cryptographic verification
No trusted verifier
One plaintext lookup

VSM = UNS + SMPP

Two modules separate slot allocation from anonymous placement



UNS - very simple instantiation

For a pair of users

- Each honest user selects a slot uniformly at random.



Case 1: different slots (the desired outcome)

UNS - very simple instantiation

For a pair of users

- Each honest user selects a slot uniformly at random.



Case 2: the same slot (a collision)

Test collision

Do the ciphertexts encrypt the same hidden slot?

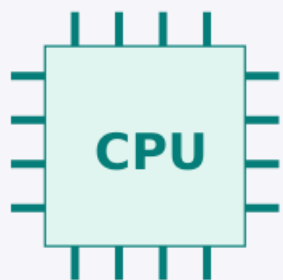


- Reveal only equal / different, not the hidden slot values.

$E(X; r)$ encrypts slot X with randomness r . X_1 and X_2 are the users' chosen slots.

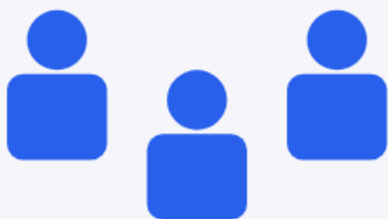
Collision testing

Two implementations of the equal / different test



Trusted hardware (TEE)

- Decrypt inside the enclave and return the equality bit.
- Keep the key and running state private. Audit the signed query chain.



MPC / plaintext-equivalence test

- Servers jointly test equality using secret-key shares.
- Privacy relies on the stated honest-server assumption.

TEE = trusted execution environment. MPC = secure multiparty computation.

UNS: advance through a candidate list

Each user posts an encrypted list of possible slots

A candidate is one possible slot, encrypted before UNS begins.

User 1

E(3)

active candidate

E(1)

E(5)

...

User 2

E(3)

active candidate

E(4)

E(2)

...

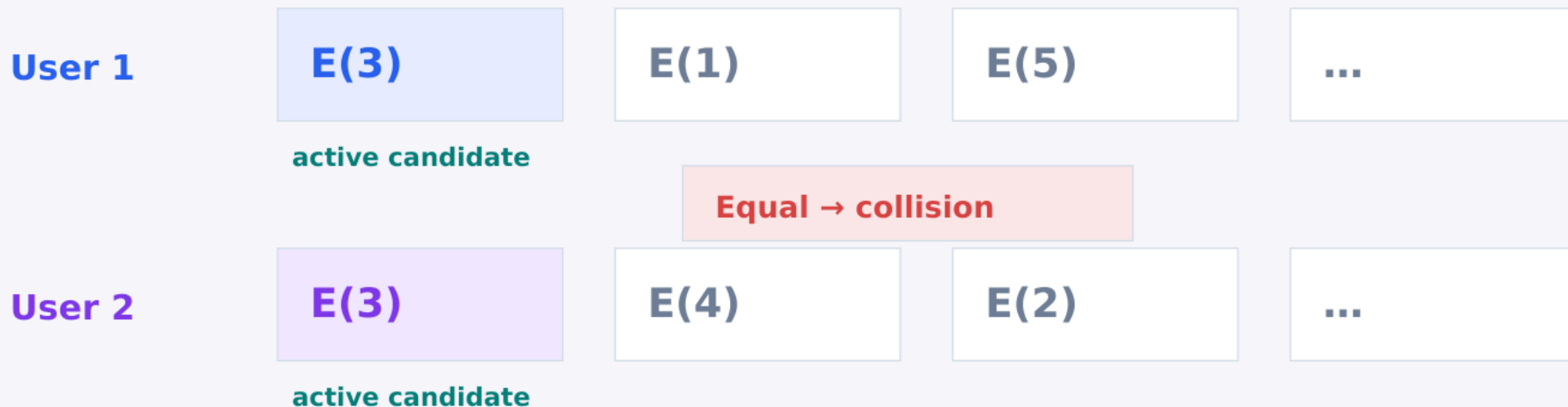
Only the active candidates are compared.

$E(x)$ denotes an encryption of slot x ; the hidden values are shown only to explain the example.

UNS: advance through a candidate list

Test the active candidates

A candidate is one possible slot, encrypted before UNS begins.



The test returns equality; it does not publish the slot.

$E(x)$ denotes an encryption of slot x ; the hidden values are shown only to explain the example.

UNS: advance through a candidate list

On collision, both users move to their next candidate

A candidate is one possible slot, encrypted before UNS begins.



Continue testing until all surviving active slots are distinct.

$E(x)$ denotes an encryption of slot x ; the hidden values are shown only to explain the example.

UNS: protect honest users from list exhaustion

Bounded candidate lists limit disruption

- Requirement: an honest user should not exhaust its candidate list.
- The probability is very small even with a logarithmic-sized list.

$$\text{Pr}[\text{any honest list is exhausted}] \leq n \cdot 4^{-K}$$

n = number of users; K = candidates per user; Pr = probability.

The bound uses a slot space of at least $4n$ and the paper's sampling assumptions.

Example: 1,000 users and 20 candidates per user give a bound below 10^{-9} .

SMPP: homomorphic aggregation

Each user contributes a validated sparse encrypted vector

	Slot 1	Slot 2	Slot 3	Slot 4	Slot 5
User 1	E(0)	E(0)	E(0)	E(M ₁)	E(0)
User 2	E(0)	E(0)	E(M ₂)	E(0)	E(0)
User 3	E(M ₃)	E(0)	E(0)	E(0)	E(0)
User 4	E(0)	E(M ₄)	E(0)	E(0)	E(0)
Encrypted output	—	—	—	—	—

Accept only well-formed vectors tied to the user's assigned slot.

SMPP: homomorphic aggregation

Add all users: one ciphertext per slot

	Slot 1	Slot 2	Slot 3	Slot 4	Slot 5
User 1	E(0)	E(0)	E(0)	E(M ₁)	E(0)
User 2	E(0)	E(0)	E(M ₂)	E(0)	E(0)
User 3	E(M ₃)	E(0)	E(0)	E(0)	E(0)
User 4	E(0)	E(M ₄)	E(0)	E(0)	E(0)
Encrypted output	E(M ₃)	E(M ₄)	E(M ₂)	E(M ₁)	E(0)

Five slots → five ciphertexts; the empty slot still contains an encryption of zero.

Somewhat Homomorphic Encryption (BGN)

Multiply entry by entry under encryption

Unit vector

$E(0)$

$E(0)$

$E(1)$

$E(0)$

× entry-wise homomorphic multiplication

Any message vector

$E(a)$

$E(b)$

$E(M)$

$E(d)$

↓ only the allocated slot survives

Masked output

$E'(0)$

$E'(0)$

$E'(M)$

$E'(0)$

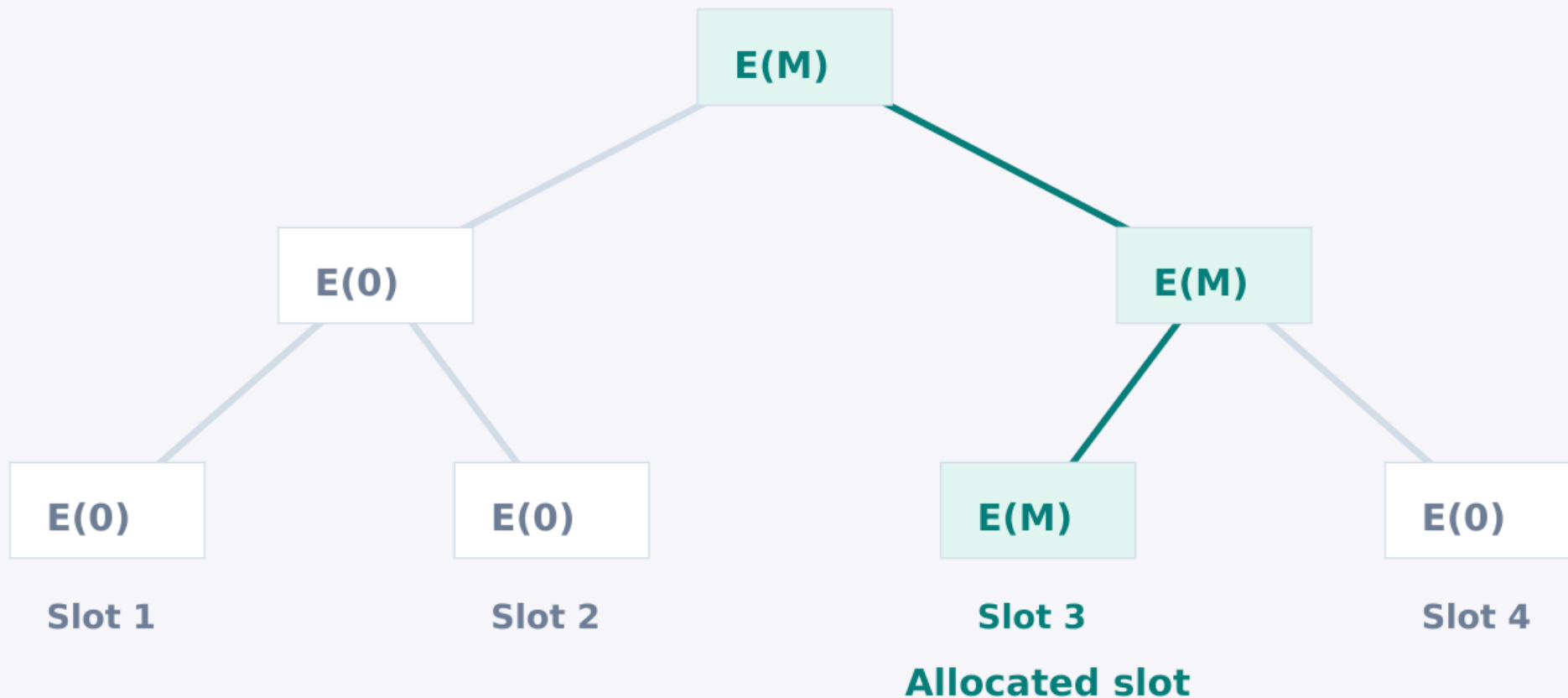
- No zero-knowledge proof that the message vector is sparse.

Validate the unit vector using homomorphic checks. a, b, d are arbitrary values; M is the message.

Boneh-Goh-Nissim. E = input encryption; E' = encryption after multiplication.

FHE: homomorphic traversal

Goal: route an encrypted message to its allocated hidden slot

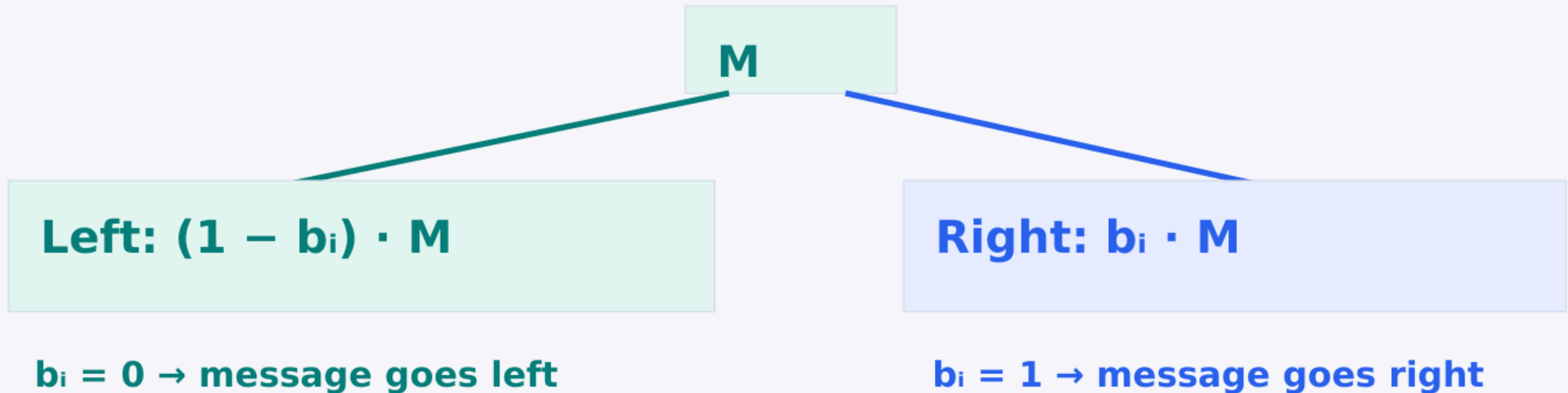


Encrypted address bits control the path without revealing it.

Every other output slot contains $E(0)$. E = encryption; M = message.

Homomorphic traversal

- Decompose the allocated location's zero-based address into bits.
- At each level, evaluate its control bit on both branches.



M = message; b_i = binary address bit at tree level i ; i = level index.

The unselected branch is zero. Apply the same rule at every level.

FHE - control pass

Use an encrypted control bit to route an encrypted message

Control bit b_i

Left: $(1 - b_i) \odot E(M)$

Right: $b_i \odot E(M)$

A plaintext bit scales the ciphertext.

Encrypted control bit $E(b_i)$

Left: $(E(1) \ominus E(b_i)) \otimes E(M)$

Right: $E(b_i) \otimes E(M)$

The bit and the message remain hidden.

$$b_i \rightarrow E(b_i)$$

E = encryption; M = message; b_i = address bit at level i ; i = level index.

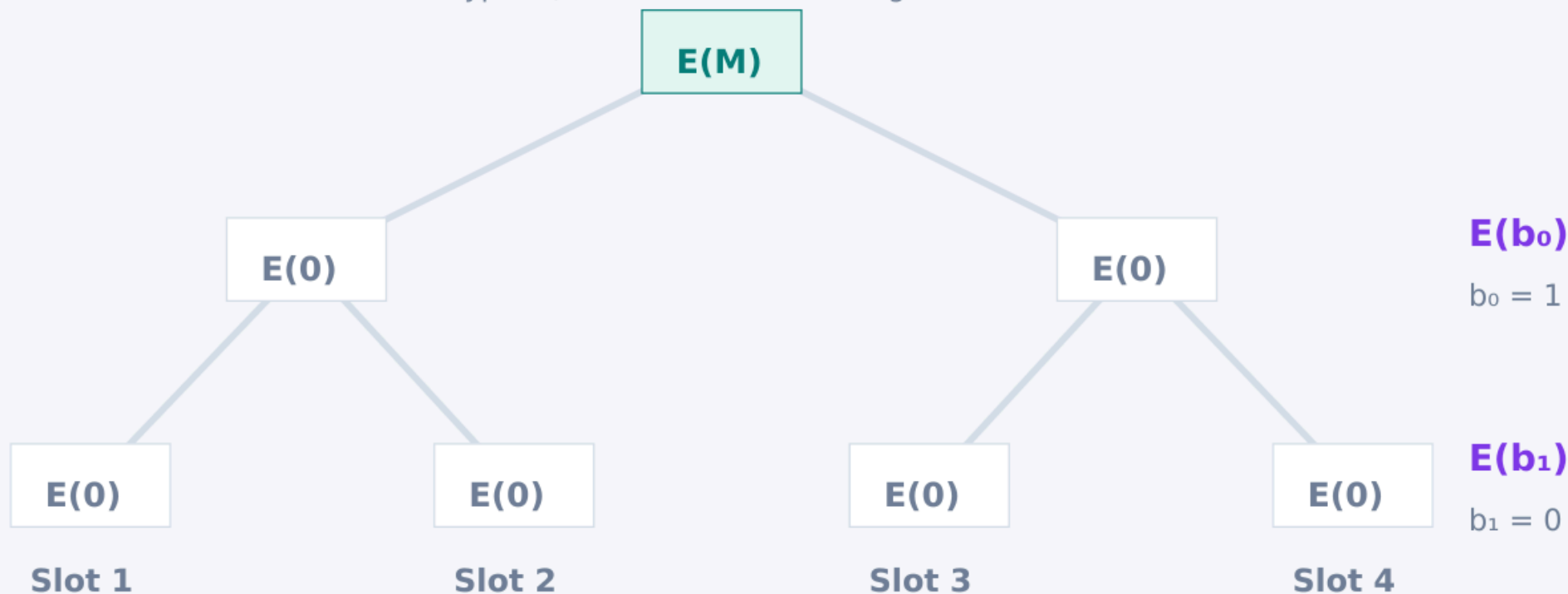
$\odot$ = plaintext-ciphertext multiplication; $\ominus$ and $\otimes$ = homomorphic subtraction and multiplication.

FHE: route a message using encrypted control bits

Upload an encrypted message and encrypted bits of its slot

Encrypted slot $E(3)$ $\rightarrow$ zero-based address $E(2)$ $\rightarrow$ $\langle E(1), E(0) \rangle$

Slots start at 1; address 2 = 10_2 . E denotes encryption; bits are shown most significant first.



M = message; E = encryption; b_0 and b_1 = first and second binary control bits.

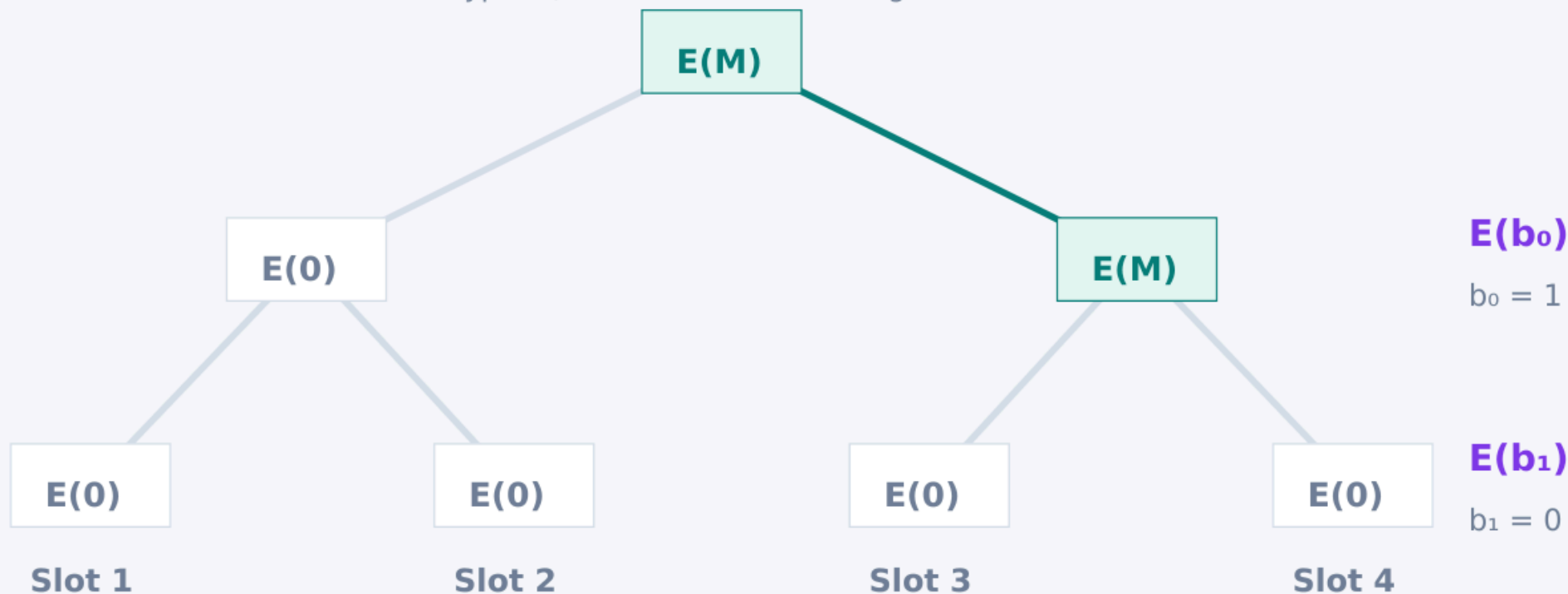
The server evaluates both branches under encryption, without learning the path.

FHE: route a message using encrypted control bits

First control bit selects the right subtree

Encrypted slot $E(3)$ $\rightarrow$ zero-based address $E(2) \rightarrow \langle E(1), E(0) \rangle$

Slots start at 1; address 2 = 10_2 . E denotes encryption; bits are shown most significant first.



M = message; E = encryption; b_0 and b_1 = first and second binary control bits.

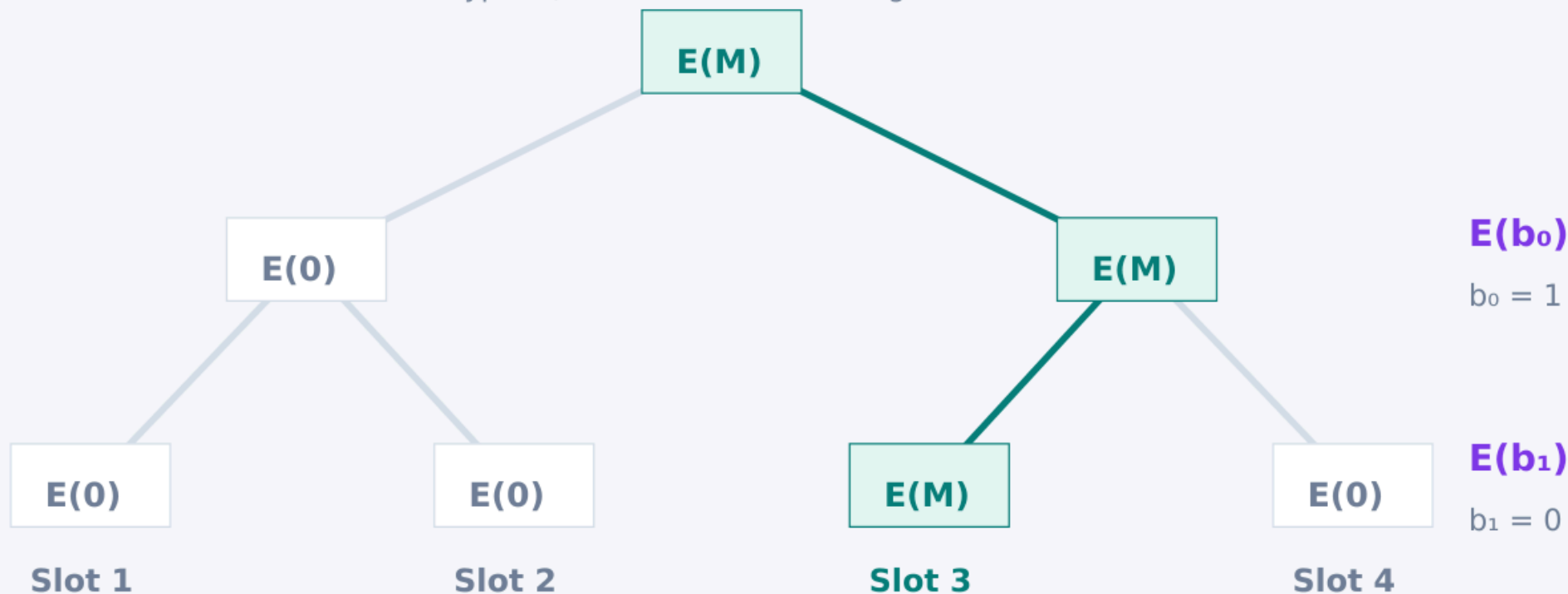
The server evaluates both branches under encryption, without learning the path.

FHE: route a message using encrypted control bits

Second control bit selects the left leaf

Encrypted slot $E(3)$ $\rightarrow$ zero-based address $E(2)$ $\rightarrow$ $\langle E(1), E(0) \rangle$

Slots start at 1; address 2 = 10_2 . E denotes encryption; bits are shown most significant first.



M = message; E = encryption; b_0 and b_1 = first and second binary control bits.

Every other leaf encrypts zero; only the allocated slot retains $E(M)$.

FHE: encrypted unit vector $\times$ encrypted message

All other leaves encrypt zero

Encrypted unit vector

$E(0)$

$E(0)$

$E(1)$

$E(0)$

$\times$ broadcast the same encrypted message to every entry

Same encrypted M

$E(M)$

$E(M)$

$E(M)$

$E(M)$

$\downarrow$ evaluate the products homomorphically

Encrypted output

$E(0)$

$E(0)$

$E(M)$

$E(0)$

M is constant across the vector; the encrypted unit vector selects its location.

E denotes encryption; M is the message. $\times$ means entry-wise homomorphic multiplication.

Three SMPP instantiations

One plaintext payload element per user

Instantiation	Client upload	Public work	Assumption / note
ElGamal + NIZK	$O(m)$	$O(nm)$ group mults	Threshold encryption; per-user proofs
BGN + checks	$O(m)$	$O(nm)$ pairings / group ops	Honest-majority setup MPC
FHE traversal	$\tilde{O}(\log m + 1)$	$O(m)$ mults per user	Theoretical compression; FHE evaluation + proofs

FHE total batch work: $O(nm)$ homomorphic multiplications.

n = users; m = slots. One payload element must fit each scheme's plaintext space.

Security statement

What is guaranteed, and under which model

Anonymity

A PPT adversary cannot link honest users to hidden slots or messages beyond the final output multiset.

Integrity

Every honest user's valid message appears at its assigned slot, or the protocol aborts.

Self-verifiability

After SMPP-Dec, an honest user checks the plaintext at its own secret slot.

Model assumptions

- Static malicious model; adversary is probabilistic polynomial time (PPT).
- At least two honest users; threshold/PET privacy assumes at least one honest server. BGN setup additionally assumes an honest majority.
- Bulletin board is authenticated and append-only. Signed acknowledgments make omission of an acknowledged post publicly attributable; otherwise omission is board-level censorship.

Possible next steps and open questions

More efficient UNS

Reduce equality tests and interaction while retaining hidden unique slots and a small trusted state.

Concrete implementation

Measure client upload, collision-testing cost, aggregation time and threshold opening.

Less communication

Seek sublinear client upload without the full cost of FHE evaluation and setup.

Stronger models

Study adaptive corruption, dynamic participation and unlinkability across epochs.

Summary

VSM benefits

- Self-verifiability: after decryption, each user checks their own slot with no cryptographic computation.
- Non-interactive SMPP: users post independently after UNS, and anyone can compute the encrypted output table.
- Integrity: malformed inputs are rejected or detected, protecting honest users' assigned slots.
- Limited trust: trusted slot allocation is confined to UNS. SMPP needs no trusted online mixer.
- Reusable UNS: run SMPP many times with the same slots, preserving privacy but not cross-batch unlinkability.

Setup, key protection and optional threshold decryption retain their stated assumptions.

Takeaways

1

Self-mix

Users receive private unique slots and place their own encrypted messages.

2

Modularity

UNS handles uniqueness; SMPP handles anonymous placement; trust assumptions are swappable.

3

Verification

After opening, each sender verifies inclusion with one plaintext lookup.

Questions?

doronz@gmail.com

Backup: closest conceptual prior art

Lead with the property—not with a “first” claim

VSM sender-side check

$$T[\lambda_i] = M_i$$

- One plaintext table lookup
- No cryptographic proof verification by the sender
- No trusted verifier needed for the inclusion check

Unconditional verification $\neq$ unconditional privacy

Closest conceptual prior art

sElect / Selene / Hyperion

Private tracker / plaintext lookup style verification

VSM's architectural delta

Anonymous communication over a bulletin board; slot assignment separated from placement; no mixnet in the placement path; deterministic public aggregation

Not claimed: receipt-freeness or coercion resistance

Backup: security games at a glance

Anon-UNS

- Challenger samples honest candidate lists.
- Two honest challenge users are swapped between target slots.
- Adversary receives the public equality-test transcript and encrypted outputs.
- Goal: guess which user received which challenge slot.

Anon-SMPP

- Adversary chooses two honest senders and two messages.
- Challenger chooses two distinct slots and swaps the challenge messages.
- Adversary sees the public SMPP transcript and encrypted output table.
- Goal: distinguish which sender was associated with which message / slot.

Backup: exact aggregation counts

One plaintext payload element per user

ElGamal SMPP

$m(k - 1)$ ciphertext multiplications

$2m(k - 1)$ group multiplications

BGN SMPP

mk pairings + $m(k - 1)$ target-group multiplications

FHE traversal

$2(1 + 1)(m - 1) = 4(m - 1)$ multiplications per user

One unit-vector traversal and one payload traversal; depth $\lceil \log_2 m \rceil$.

n = users; k = accepted users ($k \leq n$); m = slots. FHE count assumes a full tree with m leaves.